

CHAPITRE I – BASE DE DONNEES REPARTIE

1. Introduction à la répartition

Une base de données centralisée est gérée par un seul SGBD, est stockée dans sa totalité à un emplacement physique unique et ses divers traitements sont confiés à une seule et même unité de traitement. Par opposition, une base de données distribuée est gérée par plusieurs processeurs, sites ou SGBD. Les bases de données réparties, fédérées et parallèles sont trois domaines distincts mais qui ont une problématique commune, la distribution des données ou des traitements sur plusieurs unités qui coopèrent pour gérer les informations. Les objectifs de la distribution de données sont multiples :

- Limiter le transfert d'information (en nombre et en volume) en répartissant les données les plus utilisées. Ceci est important pour une BD dont les utilisateurs sont géographiquement dispersés.
- Accroître les performances par la répartition de la charge de travail sur plusieurs unités de traitement opérant en parallèle.
- Augmenter la fiabilité en faisant effectuer le même traitement par plusieurs ordinateurs et en dupliquant les données sur différents sites (Etendre la disponibilité des informations), lorsque la BD concerne un domaine sensible où la moindre erreur peut avoir des conséquences catastrophiques.
- Faire croître en souplesse une BD ou même tout le système de base de données. Le but est ici de ne pas modifier l'existant lorsque l'on désire transformer la structure des informations (évolution du schéma conceptuel), utiliser un nouveau support ou une nouvelle technologie pour le stockage en mémoire secondaire, ou plus radicalement utiliser un nouveau SGBD.
- Fusionner en douceur deux systèmes d'information en faisant coopérer leurs bases de données. Le principe est ici de maintenir les applications existantes tout en permettant l'interopérabilité des deux systèmes d'information.

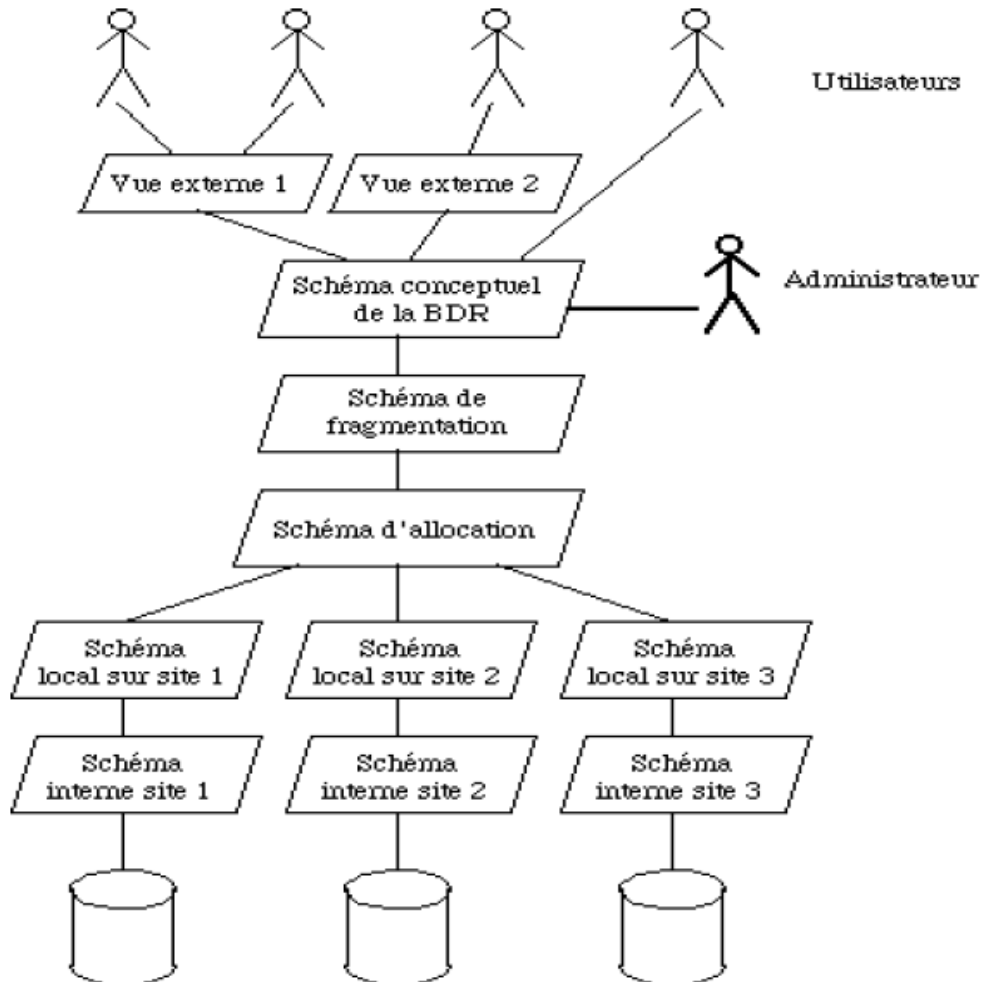
2. Principes d'une Base de données répartie

Une base de données répartie (BDR) est une base de données dont différentes parties sont stockées sur des sites, généralement géographiquement distants, reliés par un réseau. La réunion de ces parties forme la base de données répartie. Un système de bases de données réparties ne doit donc en aucun cas être confondu avec un système dans lequel les bases de données sont accessibles à distance (selon le principe client serveur). Il ne doit non plus être confondu avec un système multi base ou chaque utilisateur accède à différentes bases de données en spécifiant leur nom et adresse (Plusieurs BD hétérogènes), et le système se comporte alors comme un serveur de BD et n'apporte aucune fonctionnalité particulière à la répartition. Au contraire, un système de bases de données réparties est suffisamment complet pour décharger les utilisateurs de tous les problèmes de concurrence, fiabilité, optimisation de requêtes ou transaction sur des données gérées par différents SGBD sur plusieurs sites. Par exemple, une banque peut posséder des agences dans plusieurs villes. Dans une BD centralisée, le siège social de la banque gèrerait tous les comptes des clients et les agences devraient communiquer avec le siège social pour avoir accès aux données. Dans une BD répartie, les informations sur les comptes sont distribuées dans les agences et celles-ci sont interconnectées (entièrement ou partiellement) afin qu'elles puissent avoir accès aux données externes. Cependant, la répartition de la base de données bancaire est invisible aux agences en tant qu'utilisateurs, et la seule conséquence directe pour elles est que l'accès à certaines données est beaucoup plus rapide.

2.1. Utilisation d'une BD répartie

Le principe fondamental des BDR est la transparence pour l'utilisateur. Cette transparence s'exprime sous trois formes :

Les utilisateurs accèdent à la BD soit directement par le schéma conceptuel soit indirectement au travers de vues externes. Mais en aucun cas ils n'ont les moyens d'accéder aux schémas locaux ni de préciser le site. C'est le principe de *transparence de localisation*. Dans l'exemple précédent, un client peut ouvrir un compte à TNR et effectuer régulièrement des opérations à FNR. C'est le système qui recherche le site où sont mémorisés ses informations et non l'utilisateur qui doit l'indiquer. De même, les utilisateurs n'ont pas à connaître les partitionnements de la base de données. C'est le principe de *transparence de partitionnement*. Les utilisateurs ne doivent pas savoir si telle information est fractionnée, et ne doivent donc pas se préoccuper de la réunifier. C'est le système qui gère les partitionnements et les modifie en fonction de ses besoins, et c'est donc lui qui doit rechercher toutes les partitions et les intégrer en une seule information logique présentée à l'utilisateur.



Architecture de schéma d'une base de données répartie

Enfin, les utilisateurs n'ont pas à savoir si plusieurs copies d'une même information sont disponibles. C'est le principe de *transparence de duplication*. La conséquence directe est que lors de la modification d'une information, c'est le système qui doit se préoccuper de mettre à jour toutes les copies.

Dans l'exemple précédent, il est fort possible que lorsqu'un client possède des comptes (courant, épargne logement, ...) à TNR mais effectue régulièrement des retraits d'argent à FNR, les informations le concernant soient réparties entre TNR à FNR : l'historique des opérations du compte courant est conservé à FNR, la gestion de ses autres comptes est assurée à TNR, et le solde du compte courant est dupliqué à TNR et FNR.

Dans une BDR, les sites ne sont donc pas autonomes, bien que chaque site possède une machine avec un SGBD et une partie de la base de données, et pourraient effectuer certains traitements localement. En effet, les utilisateurs accèdent aux données par l'intermédiaire du schéma conceptuel global et non par l'intermédiaire du schéma d'un site, et c'est uniquement le système qui désigne le ou les sites qui sont concernés par la demande de

l'utilisateur. L'indépendance physique, assurée dans les bases de données traditionnelles, est donc conservée dans les BDR.

2.2. Niveau de répartition

La répartition d'une base de données intervient dans les trois niveaux de son architecture en plus de la répartition physique des données :

- Niveau externe : les vues (schémas externes) sont distribuées aux utilisateurs sur leurs sites, les *sites utilisateurs*.
- Niveau conceptuel : le schéma conceptuel des données (schéma logique) est associé, par l'intermédiaire du schéma de répartition (lui même décomposé en un schéma de fragmentation et un schéma d'allocation), aux schémas locaux qui sont réparties sur plusieurs sites, les *sites physiques*.
- Niveau interne : le schéma interne global n'a pas d'existence réelle mais fait place à des schémas internes locaux répartis sur différents sites (en principe les sites d'accueil des schémas logiques répartis). Tous les niveaux doivent être concernés par la répartition pour que l'on puisse parler d'une vrai BD répartie. Sinon on n'a affaire qu'à des clients éloignés (répartition externe) ou à un stockage physique multi machines (répartition interne). On doit aussi remarquer que la distribution de la base de donnée se fait plus ou moins conformément à la répartition logique des utilisateurs dans l'entreprise (en divisions, services, instituts, ...) et à la répartition physique des moyens informatiques dans l'entreprise (en centres de calculs, salles des machines, ordinateurs).

3. Conception de bases de données réparties (distribuées)

L'utilisation d'une BD réparti peut naître de deux approches distinctes ; soit la nécessité de connecter les différentes bases de données existantes, soit le besoin de disponibilité de données nécessaires à la globalisation des systèmes d'information.

Conception descendante : À partir d'un schéma global, on construit les schémas locaux. Cette conception se prête pour la mise en place de nouvelles bases de données à WAN.

Conception ascendante : La conception ascendante permet l'intégration de bases de données locales existantes dans une base de données distribuée. Il s'agit cette fois de construire un schéma global à partir de schémas locaux existants. La plus difficile à mettre en place. La fragmentation consiste à isoler les sous relations placée sur un même site à partir d'une relation globale.

- Sélection de lignes à fragmentation Horizontale (UNION)
- Projection de colonnes à fragmentation Verticale

La fragmentation verticale est le découpage d'une table en sous tables par projection permettant de sélectionner les colonnes composant chaque fragment. La relation initiale doit pouvoir être recomposée par jointure des fragments.

Exemple :

```
CREATE VIEW v_commande AS
SELECT c1.numero, c1.clients, c2.produit, c2.qte FROM commande1@site1
c1, commande2@site2 c2 WHERE c1.numero=c2.numero;
```

La fragmentation horizontale est un découpage d'une table en sous tables par utilisation de prédicats permettant de sélectionner les lignes appartenant à chaque fragment. La relation initiale peut être retrouvée par UNION des fragments.

Exemple :

```
CREATE VIEW v_clients AS
SELECT numero, nom, localite FROM clients1@site1
UNION
```

```
SELECT numero, nom, localite FROM client2@site2;
```

La fragmentation mixte résulte de l'application successive d'opérations de fragmentation horizontale et verticale sur une relation globale. Chaque fragment est placé sur un site. Un schéma doit être élaboré afin de déterminer la localisation de chaque fragment et sa position dans le schéma global. Pour fragmenter les requêtes, il est nécessaire de connaître les règles de localisation des données. Lors de l'exécution d'une requête le SGBDR doit décomposer la requête globale en sous requêtes locales en utilisant le schéma de répartition (clé de répartition).

4. Gestion des données réparties

Les règles d'exécution et les méthodes d'optimisation de requêtes définies pour un contexte centralisé sont toujours valables, mais il faut maintenant prendre en compte d'une part la fragmentation et la répartition des données sur différents sites et d'autre part le problème du coût des communications entre sites pour transférer les données. Le problème de la fragmentation avec ou sans duplication concerne principalement les mises à jours tandis que le problème des coûts des communications concerne surtout les requêtes.

4.1. Mise à jour de BD réparties

La principale difficulté réside dans le fait qu'une mise à jour dans une relation du schéma global se traduit par plusieurs mises à jour dans différents fragments. Il faut donc identifier les fragments concernés par l'opération de mise à jour, puis décomposer en conséquence l'opération en un ensemble d'opération de mise à jour sur ces fragments.

4.1.1. Insertion

Retrouver le fragment horizontal concerné en utilisant les conjonctions de condition qui définissent les fragments horizontaux. Insertion du tuple dans tous les fragments verticaux correspondants.

4.1.2. Suppression

Rechercher le tuple dans les fragments qui sont susceptibles de contenir le tuple concerné (en utilisant, dans la mesure du possible les conjonctions de condition). Supprimer les valeurs d'attribut du tuple dans tous les fragments verticaux.

4.1.3. Modification

Rechercher le(s) tuple(s) dans les fragments qui sont susceptibles de contenir ce(s) tuple(s) (en utilisant, dans la mesure du possible les conjonctions de condition). Modifier la (les) valeur(s) d'attribut du (des) tuple(s). Vérifier que le(s) nouveau(x) tuple(s) ainsi produit vérifie (nt) toujours la conjonction de condition qui définit le fragment horizontal dans lequel se trouve (nt) ce(s) tuple(s), et si ce n'est pas le cas, déplacer le(s) tuple(s) concerné(s) (suppression puis insertion) dans le bon fragment horizontal.

4.2. Requêtes sur BD réparties

Comme pour le traitement de requêtes en Bases de données centralisées, on produit l'arbre algébrique de la requête que l'on optimise avec les mêmes critères. Chaque feuille de l'arbre représente une relation, et chaque nœud représente une opération algébrique (opérateur + opérandes). Cette requête algébrique optimisée est ensuite enrichie avec les informations sur la répartition des données sur les différents sites, et sur le lieu (site) où chaque opération de la requête doit être exécutée. Le temps total d'exécution d'une requête tient compte du temps de calcul CPU nécessaire pour exécuter les opérations algébriques (jointures, sélections ...), mais aussi et surtout du temps de transfert entre les sites des données nécessaires à la requête.

4.2.1. Transferts de données

Le temps de transmission d'un message tient compte du temps d'accès et de la durée de la transmission (volume des données / débit de transmission). Le temps d'accès est négligeable sur un réseau local, mais peut atteindre quelques secondes pour des transmissions sur de longues distances ou via satellite. Dans ces conditions, un

traitement ensembliste des données s'impose. L'unité de transfert entre sites est donc une relation ou un fragment, et non une occurrence.

4.2.2. Traitement de requêtes réparties

Le but est d'affecter de manière optimale un site d'exécution pour chacune des opérations algébriques de l'arbre. Pour cela, on associe à chacune des feuilles le site sur lequel la relation va être puisée. Lorsqu'une relation est dupliquée, le choix du site de départ est un élément d'optimisation (la décision peut alors être laissée en suspend). On cherche ensuite à associer à chaque nœud de l'arbre le site sur lequel l'opération algébrique associée à ce nœud sera exécutée. Généralement, il faut faire localement tous les traitements qui peuvent y être faits. Ainsi, lorsque tous les opérandes d'une même opération algébrique sont situés sur le même site, la solution la moins coûteuse pour exécuter cette opération est le plus souvent de l'exécuter sur ce site. Pour diminuer le volume de données transmis d'un site à un autre, il faut limiter les transferts d'information aux seules informations utiles. Pour cela il faut systématiquement rajouter des projections dans l'arbre algébrique pour abandonner les attributs inutiles. Il faut aussi noter que les parties de requêtes indépendantes peuvent être exécutées en parallèle sur des sites différents et donc abaisser le temps total d'exécution.

4.2.3. Optimisation dynamique des requêtes

La principale différence entre un SGBD et un SGBDR au point de vue de l'optimisation des requêtes se trouve dans le fait que l'optimisation de requête en centralisé est principalement statique, c'est à dire calculé avant l'exécution en se basant sur les critères de volume de données, pour savoir si telle opération peut être effectuée en mémoire centrale, et moyens d'accès (index, ordonnancement, tri). Au contraire, l'optimisation de requête dans un environnement réparti se base aussi sur le temps de communication qui est fonction des critères suivants: volume de données transféré, nombre de communications, temps d'accès et temps de réponse, temps de calcul UC sur chacun des sites.

Elle peut donc être fortement dynamique: après avoir initialement généré un arbre de requête, la stratégie adoptée pour l'exécution est ascendante. C'est à dire que l'affectation de chaque nœud de l'arbre à un site peut être décidée en cours d'exécution en fonction des différents volumes de données intermédiaires obtenus sur les sites. On part donc des feuilles, où l'on connaît les données (type, volume, statistiques sur la répartition des occurrences dans les domaines de valeurs...) pour remonter au niveau supérieur et prendre une décision sur le site d'affectation de l'opérateur. Et ainsi de suite pour les niveaux supérieurs jusqu'à la racine. Il faut en effet prendre en compte tout l'arbre de la requête pour regarder si une décision d'affectation qui semble bonne ponctuellement ne risquerait pas de poser des problèmes aux niveaux supérieurs. D'une manière générale, il faut tenir compte des volumes de données de chaque relation, et de leur composition. Il est en effet parfois intéressant de tenir compte du calcul effectué par un site avant que les autres sites se mettent à travailler.

4.2.4. Semi jointure

Les BDR ont abouti à la définition d'un nouvel opérateur, la semi jointure, qu'il est parfois intéressant d'utiliser. Il s'agit en fait d'une double jointure : le principe est d'effectuer deux petites jointures plutôt qu'une grosse; c'est à dire deux petites transmissions de données plutôt qu'une seule beaucoup plus volumineuse. Soit R et S deux relations ayant X comme ensemble d'attributs en commun. La semi jointure entre R et S est $R \bowtie_X S$. C'est à dire que l'on ne garde dans R que les occurrences ayant pour X une valeur qui est présente dans au moins une occurrence de S.